



Documentação Técnica: Sistema de Autenticação com Listagem Paginada

Este documento explica detalhadamente a arquitetura, o funcionamento do código, os conceitos do React/Node utilizados e o passo a passo para recriar e rodar o projeto do zero.

1. Como o Projeto foi Criado do Zero (O Passo a Passo dos Comandos)

Se fôssemos recriar a estrutura desse projeto hoje, estes seriam os comandos exatos executados no terminal Linux:



No Frontend (Interface do Usuário)

Para iniciar a estrutura do React moderna e ultra-rápida, o comando inicial é o do Vite:

```
# 1. Cria a pasta e a estrutura base do React com Vite  
npm create vite@latest Frontend -- --template react
```

```
# 2. Entra na pasta criada  
cd Frontend
```

```
# 3. Instala as dependências padrão do projeto  
npm install
```

```
# 4. Instala as bibliotecas específicas que usamos no formulário e nas requisições  
npm install react-hook-form axios
```

```
# 5. Roda o projeto em ambiente de desenvolvimento local (disponível em http://localhost:5173)  
npm run dev
```



No Backend (Servidor e Banco de Dados)

Para criar o servidor que processa os dados e conversa com o banco de dados:

```
# 1. Cria a pasta do servidor e entra nela  
mkdir Backend  
cd Backend
```

```
# 2. Inicia o arquivo de configuração padrão do Node (package.json)  
npm init -y
```

```
# 3. Instala o Express (servidor), Cors (segurança) e o Driver do SQLite3 (banco de dados)
npm install express cors sqlite3@5.1.6
```

```
# 4. Inicia o servidor localmente (configurado no script para rodar em http://localhost:3000)
npm start
```

2. Explicando os Conceitos do React Utilizados (O "Porquê" de Cada Coisa)

Dentro do seu arquivo App.jsx, usamos recursos nativos e bibliotecas do React para fazer a mágica acontecer. Aqui está a explicação de cada um deles:

useState (Gerenciamento de Estado Local)

O React não atualiza a tela sozinho se você mudar uma variável comum. O useState é uma função especial que serve para criar uma **variável de estado**. Quando o valor dessa variável muda, o React "redesenha" a tela automaticamente com o dado novo.

- **No seu projeto:** Nós usamos o `const [temaEscuro, setTemaEscuro] = useState(true);`. Ele guarda se o site está no modo preto (true) ou claro (false). Quando você clica no botão, o `setTemaEscuro` inverte esse valor e o React troca todas as cores da tela instantaneamente.

useContext (Contexto Global)

Por padrão, para passar um dado de uma tela para outra no React, você precisa enviar via propriedades de pai para filho (Props). Se o projeto cresce, isso vira uma bagunça. O useContext cria uma "nuvem" de dados em volta do projeto (o AuthContext).

- **No seu projeto:** Ele permite que o App.jsx pegue a lista de usuários, as mensagens de erro e a função de cadastrar diretamente da central do projeto (AuthContext), sem precisar fazer gambiarras de importação.

useForm (Biblioteca react-hook-form)

Gerenciar formulários no React na mão exige criar um useState para cada input (Nome, E-mail, Senha...). Fica um código gigante. O useForm automatiza isso.

- **O register:** Vincula o input do HTML à biblioteca.
- **O handleSubmit:** Valida se tudo está preenchido corretamente antes de disparar a função de salvar.
- **O watch:** Fica "olhando" o input da senha em tempo real para permitir que a validação de "Confirme a Senha" saiba se as duas são iguais.
- **O errors:** Guarda quais campos falharam na validação para exibir o texto vermelho na tela.

3. Explicação de Todas as Funções do Projeto

O sistema é dividido em duas partes que se comunicam através de requisições HTTP (com a biblioteca Axios).

Funções do Frontend (App.jsx e AuthContext)

1. **salvarFormulario(dados):**

- É disparada quando o usuário clica em "Finalizar Cadastro" e passa por todas as validações de campo.
- Ela chama a função global cadastrarUsuario(dados) e, se o banco de dados responder que deu certo, ela roda o reset(), que limpa todos os campos do formulário automaticamente.

2. **cadastroUsuario(dados) (Dentro do Contexto):**

- Pega o Nome, E-mail e Senha e faz um disparo assíncrono via axios.post para o link do backend (/api/cadastro).
- Se o e-mail já existir no banco, ela captura o erro e joga na tela a mensagem de erro. Se der certo, ela coloca o aviso verde de sucesso e atualiza a tabela chamando a busca de usuários.

3. **buscarUsuariosPaginados(pagina):**

- Faz um disparo via axios.get para o endpoint /api/usuarios, enviando na URL a página que quer ler (ex: ?pagina=1&limite=5).
- Ela recebe do servidor a lista com apenas os 5 usuários daquela página específica e o cálculo de quantas páginas existem no total, atualizando os estados da paginação na tela.

Funções do Backend (server.js)

O servidor Express funciona como um "garçom". Ele fica esperando as requisições chegarem em rotas (endpoints) específicas e executa comandos SQL dentro do banco SQLite:

1. **Configuração do CORS (app.use(cors(...))):**

- Uma regra de segurança dos navegadores impede que um site (Vercel) converse com um servidor de outro link (Render). O CORS é a função que diz ao servidor: *"Pode aceitar conexões vindas do link da Vercel, eu confio nelas"*. Nós configuramos com o caractere coringa * para aceitar qualquer subdomínio de teste gerado pela Vercel.

2. **Rota de Cadastro (app.post('/api/cadastro')):**

- Recebe os dados do formulário.
- Executa o comando SQL: INSERT INTO usuarios (nome, email, senha) VALUES (?, ?, ?).
- Possui um tratamento de erro estruturado: se o SQLite devolver um erro de UNIQUE constraint, o servidor responde com o status 400 avisando que o e-mail já está cadastrado.

3. **Rota de Listagem Paginada (app.get('/api/usuarios')):**

- Lê os parâmetros passados na URL (pagina e limite).
- Faz um cálculo matemático de deslocamento (Offset) para saber quais registros pular. Se estamos na página 2 com limite de 5, o Offset é 5 (ele pula os 5 primeiros e pega do 6 ao 10).
- Executa o comando SQL: SELECT id, nome, email FROM usuarios LIMIT ? OFFSET ?.
- Faz um SELECT COUNT(*) para saber o total de usuários cadastrados e calcular

quantas páginas totais existem, devolvendo tudo mastigado para o React.

4. Como Executar o Projeto Localmente (Para testes rápidos)

Se você ou a professora clonarem o repositório no GitHub para rodar na máquina:

1. Iniciar o Backend:

```
cd Backend  
npm install  
npm start
```

(O arquivo `banco.db` será criado na hora automaticamente e o servidor rodará na porta 3000).

2. **Iniciar o Frontend:**

```
``bash  
# Em outro terminal  
cd Frontend  
npm install  
npm run dev
```

(A interface abrirá no endereço local da porta 5173).